บทที่ 2: โมเดลข้อมูลและภาษาสำหรับสืบค้นข้อมูล

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 15 นาที และตัดคำว่า [หยุด] ออกแล้ว Edge จะได้ไม่อ่านออกมา)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่สอง ว่าด้วยโมเดลข้อมูลและภาษาสำหรับสืบค้นข้อมูล

โมเดลข้อมูล คือวิธีที่เราใช้มองและจัดรูปแบบข้อมูล ผู้เขียนบอกว่าโมเดลข้อมูลเป็นส่วนที่สำคัญที่สุดอย่างหนึ่งในการพัฒนาซอฟต์แวร์ เพราะไม่ได้มีผลแค่วิธีเขียนโค้ด แต่มีผลไปถึงวิธีคิดเรื่องปัญหาที่เรากำลังแก้ด้วย

ซอฟต์แวร์ส่วนใหญ่สร้างขึ้นเป็นชั้น ๆ ซ้อนกัน ชั้นบนสุดคือโลกจริง เช่น คน สินค้า หรือคำสั่งซื้อ นักพัฒนาแปลงสิ่งเหล่านี้เป็นออบเจกต์ในโค้ด แล้วเก็บลงฐานข้อมูลในรูปแบบตาราง เอกสาร หรือกราฟ จากนั้นฐานข้อมูลก็แปลงข้อมูลต่อเป็นไบต์บนดิสก์ แต่ละชั้นซ่อนความซับซ้อนของชั้นที่อยู่ข้างล่างเอาไว้

ในบทนี้เราจะรู้จักโมเดลข้อมูลหลักสามแบบ ได้แก่ โมเดลเชิงสัมพันธ์ โมเดลเอกสาร และโมเดลกราฟ แล้วปิดท้ายด้วยภาษาที่ใช้สืบค้นข้อมูลแต่ละแบบครับ

ตอนที่ 2: โมเดลเชิงสัมพันธ์ ปะทะ โมเดลเอกสาร

โมเดลเชิงสัมพันธ์ หรือ Relational Model

โมเดลนี้เสนอโดย Edgar Codd ในปี 1970 แนวคิดคือเก็บข้อมูลเป็นตาราง แต่ละตารางมีหลายแถว ภาษาที่ใช้คือ SQL ซึ่งเป็นที่รู้จักกันดี

ตอนแรกหลายคนสงสัยว่าโมเดลนี้จะทำงานได้เร็วพอหรือเปล่า แต่สุดท้ายโมเดลเชิงสัมพันธ์ก็ครองตลาดมาได้ราว 30 ปี ตลอดช่วงนั้นมีโมเดลอื่นพยายามท้าชิงหลายรอบ แต่ไม่มีโมเดลไหนล้มได้

การมาของ NoSQL

ช่วงปี 2010 เริ่มมีกระแสที่เรียกว่า NoSQL ภายหลังมีการตีความชื่อนี้ใหม่ว่า Not Only SQL ซึ่งแปลว่า ไม่ได้มีแค่ SQL

กระแสนี้เกิดจากหลายสาเหตุ เช่น

- ต้องการขยายระบบให้รองรับข้อมูลจำนวนมากหรือการเขียนข้อมูลถี่ ๆ ได้ง่ายขึ้น
- อยากใช้ซอฟต์แวร์โอเพนซอร์สที่ฟรี
- ต้องการคำสั่งสืบค้นเฉพาะทางที่โมเดลเชิงสัมพันธ์ทำได้ไม่ดี
- อยากได้โครงสร้างข้อมูลที่ยืดหยุ่นกว่าตารางที่ต้องกำหนดรูปแบบตายตัว

ผู้เขียนมองว่าในอนาคต เราจะใช้ฐานข้อมูลเชิงสัมพันธ์คู่กับฐานข้อมูลแบบอื่น โดยเลือกให้เหมาะกับงานแต่ละแบบ แนวคิดนี้เรียกว่า polyglot persistence

ปัญหาความไม่เข้ากันระหว่างออบเจกต์กับตาราง

โปรแกรมส่วนใหญ่เขียนด้วยภาษาเชิงวัตถุ แต่ฐานข้อมูลเก็บข้อมูลเป็นตาราง เมื่อจะบันทึกหรือดึงข้อมูล จึงต้องมีชั้นแปลงไปมาระหว่างสองรูปแบบ ความไม่เข้ากันนี้เรียกว่า impedance mismatch

ลองนึกถึงโปรไฟล์ในเว็บสมัครงาน เช่น LinkedIn คนหนึ่งคนมีชื่อ มีประวัติการทำงานหลายที่ มีประวัติการศึกษาหลายแห่ง และมีช่องทางติดต่อหลายทาง

ในโมเดลเชิงสัมพันธ์แบบดั้งเดิม ต้องแยกข้อมูลเหล่านี้ไปเก็บในหลายตาราง แล้วเชื่อมกันด้วยรหัสผู้ใช้ เวลาจะดึงโปรไฟล์หนึ่งโปรไฟล์ ก็ต้องดึงจากหลายตารางมารวมกัน

ส่วนในโมเดลเอกสาร เราเก็บโปรไฟล์ทั้งก้อนเป็นเอกสารเดียวในรูปแบบ JSON ซึ่งมีโครงสร้างคล้ายต้นไม้ คือมีคนหนึ่งคนเป็นราก และมีประวัติงานกับประวัติการศึกษาแตกกิ่งออกไป เวลาดึงข้อมูลก็ดึงครั้งเดียวได้ทั้งหมด ข้อดีนี้เรียกว่า locality หรือการที่ข้อมูลที่เกี่ยวข้องกันอยู่ใกล้กัน

ความสัมพันธ์แบบหลายต่อหนึ่ง และหลายต่อหลาย

ปัญหาเริ่มเกิดเมื่อข้อมูลไม่ได้เป็นต้นไม้ล้วน ๆ

ตัวอย่างเช่น ในโปรไฟล์มีช่องที่อยู่ ถ้าเราเก็บเป็นข้อความว่า กรุงเทพมหานคร ซ้ำในทุกโปรไฟล์ วันหนึ่งถ้าต้องแก้ชื่อหรือแปลเป็นภาษาอื่น ก็ต้องตามแก้ทุกที่ ทางที่ดีกว่าคือเก็บเป็นรหัส แล้วให้รหัสนั้นชี้ไปยังข้อมูลจังหวัดที่เก็บไว้ที่เดียว แนวคิดนี้เรียกว่า normalization หรือการลดข้อมูลซ้ำซ้อน

แต่พอใช้รหัสอ้างอิง ข้อมูลก็กลายเป็นความสัมพันธ์แบบหลายต่อหนึ่ง คือหลายคนอยู่จังหวัดเดียวกัน ความสัมพันธ์แบบนี้ต้องใช้การเชื่อมข้อมูลข้ามตาราง ที่เรียกว่า join ฐานข้อมูลเชิงสัมพันธ์ทำเรื่องนี้ได้ดีมาก แต่ฐานข้อมูลเอกสารหลายตัวรองรับได้ไม่ค่อยดี

ยิ่งระบบโตขึ้น ข้อมูลก็ยิ่งเชื่อมโยงกันมากขึ้น เช่น บริษัทในประวัติการทำงานอาจกลายเป็นหน้าบริษัทที่มีโลโก้ หรือมีระบบให้คนรับรองทักษะของกันและกัน ความสัมพันธ์แบบนี้เป็นแบบหลายต่อหลาย ซึ่งโมเดลเอกสารจัดการได้ยากขึ้นเรื่อย ๆ

บทเรียนจากประวัติศาสตร์

เรื่องนี้ไม่ใช่ปัญหาใหม่ ช่วงปี 1970 มีฐานข้อมูลชื่อ IMS ของ IBM ซึ่งเก็บข้อมูลเป็นต้นไม้คล้ายโมเดลเอกสารในปัจจุบัน และเจอปัญหาเรื่องความสัมพันธ์แบบหลายต่อหลายเหมือนกัน

ตอนนั้นมีทางแก้สองทาง ทางแรกคือโมเดลเครือข่าย ซึ่งให้โปรแกรมเมอร์ไล่ตามลิงก์จากข้อมูลหนึ่งไปยังอีกข้อมูลหนึ่งเอง เหมือนเดินในเขาวงกต คนเขียนโค้ดต้องจำเส้นทางเอง ระบบจึงแก้ไขได้ยากมาก

ทางที่สองคือโมเดลเชิงสัมพันธ์ ซึ่งให้เราบอกแค่ว่าต้องการข้อมูลอะไร แล้วปล่อยให้ส่วนที่เรียกว่า query optimizer เลือกเส้นทางที่ดีที่สุดให้เอง ทางนี้ชนะในที่สุด เพราะโปรแกรมเมอร์ไม่ต้องกังวลเรื่องเส้นทางเข้าถึงข้อมูลอีกต่อไป

แล้วควรเลือกแบบไหน

ถ้าข้อมูลมีโครงสร้างเป็นต้นไม้ และมักดึงมาใช้ทั้งก้อน โมเดลเอกสารเหมาะกว่า โค้ดจะเรียบง่ายกว่า

ถ้าข้อมูลมีความสัมพันธ์แบบหลายต่อหลายเยอะ โมเดลเชิงสัมพันธ์เหมาะกว่า และถ้าความเชื่อมโยงซับซ้อนมาก โมเดลกราฟจะเหมาะที่สุด

เรื่องความยืดหยุ่นของโครงสร้างข้อมูล

หลายคนเรียกฐานข้อมูลเอกสารว่า ไม่มีสคีมา หรือ schemaless แต่ผู้เขียนบอกว่าคำนี้ทำให้เข้าใจผิด ความจริงคือ ยังมีโครงสร้างอยู่ เพียงแต่โค้ดที่อ่านข้อมูลเป็นฝ่ายตีความ

ผู้เขียนเสนอคำที่แม่นยำกว่า คือ schema-on-read หมายถึงตีความโครงสร้างตอนอ่าน คล้ายการตรวจชนิดข้อมูลตอนโปรแกรมทำงานในภาษาอย่าง Python

ส่วนฐานข้อมูลเชิงสัมพันธ์ใช้แบบ schema-on-write หมายถึงตรวจโครงสร้างตั้งแต่ตอนเขียน คล้ายภาษาอย่าง Java ที่ตรวจชนิดข้อมูลตั้งแต่ตอนคอมไพล์

ตัวอย่างเช่น ถ้าเดิมเราเก็บชื่อเต็มไว้ช่องเดียว แล้วอยากแยกเป็นชื่อกับนามสกุล ในโมเดลเอกสาร เราแค่เขียนโค้ดให้รองรับเอกสารทั้งแบบเก่าและแบบใหม่ แต่ในโมเดลเชิงสัมพันธ์ ต้องสั่งแก้โครงสร้างตาราง แล้วแปลงข้อมูลเก่าทั้งหมด ซึ่งในตารางใหญ่ ๆ อาจใช้เวลานาน

สองโลกกำลังเข้าหากัน

ในระยะหลัง ฐานข้อมูลเชิงสัมพันธ์หลายตัว เช่น PostgreSQL และ MySQL เริ่มรองรับข้อมูลแบบ JSON ส่วนฐานข้อมูลเอกสารบางตัว เช่น RethinkDB ก็เริ่มรองรับการเชื่อมข้อมูลแบบ join สองโมเดลจึงค่อย ๆ เข้าใกล้กันมากขึ้น และนี่เป็นทิศทางที่ดีสำหรับนักพัฒนา

ตอนที่ 3: ภาษาสำหรับสืบค้นข้อมูล

บอกว่าต้องการอะไร หรือบอกว่าต้องทำอย่างไร

การสั่งงานคอมพิวเตอร์มีสองแนวคิดใหญ่

แนวคิดแรกคือแบบสั่งทีละขั้น หรือ imperative เราต้องบอกทุกขั้นตอน เช่น วนดูข้อมูลทีละตัว ถ้าตรงเงื่อนไขก็เก็บไว้ ภาษาโปรแกรมทั่วไปส่วนใหญ่ทำงานแบบนี้

แนวคิดที่สองคือแบบประกาศ หรือ declarative เราบอกแค่ว่าต้องการผลลัพธ์แบบไหน ระบบจะหาวิธีเอง SQL เป็นภาษาแบบนี้

แบบประกาศมีข้อดีหลายอย่าง

- สั้นกว่าและเข้าใจง่ายกว่า
- ฐานข้อมูลปรับปรุงวิธีทำงานเบื้องหลังให้เร็วขึ้นได้ โดยไม่ต้องแก้คำสั่งของเรา
- ทำงานแบบขนานบนหลายซีพียูได้ง่ายกว่า เพราะไม่ได้กำหนดลำดับขั้นตอนไว้ตายตัว

ผู้เขียนยกตัวอย่างที่เห็นภาพชัดจากการทำเว็บ ถ้าอยากเปลี่ยนสีพื้นหลังของเมนูที่ถูกเลือก การเขียน CSS ใช้บรรทัดเดียวก็พอ เพราะเป็นแบบประกาศ แต่ถ้าเขียนด้วย JavaScript แบบสั่งทีละขั้น ต้องเขียนวนหาเมนูเอง โค้ดยาวกว่ามาก แถมยังพังง่ายเมื่อหน้าเว็บเปลี่ยน

MapReduce

MapReduce เป็นรูปแบบการประมวลผลข้อมูลจำนวนมากบนหลายเครื่อง ที่ Google ทำให้เป็นที่รู้จัก ฐานข้อมูลเอกสารบางตัว เช่น MongoDB ก็รองรับแบบจำกัด

แนวคิดคือเขียนฟังก์ชันสองตัว ตัวแรกชื่อ map ใช้ดึงข้อมูลที่ต้องการออกจากแต่ละเอกสาร ตัวที่สองชื่อ reduce ใช้รวมผลลัพธ์เข้าด้วยกัน เช่น นับว่าแต่ละเดือนพบเห็นฉลามกี่ตัว

MapReduce อยู่ตรงกลางระหว่างสองแนวคิด เพราะต้องเขียนโค้ดเอง แต่ระบบเป็นคนตัดสินใจว่าจะรันโค้ดที่ไหนและเมื่อไร อย่างไรก็ตาม การเขียนฟังก์ชันสองตัวให้ทำงานเข้ากันนั้นยุ่งยาก ภายหลัง MongoDB จึงเพิ่มสิ่งที่เรียกว่า aggregation pipeline ซึ่งเป็นแบบประกาศมากกว่า สรุปคือแม้แต่ระบบ NoSQL ก็ค่อย ๆ หันกลับมาหาแนวคิดแบบ SQL

ตอนที่ 4: โมเดลกราฟ

ถ้าข้อมูลของเราเต็มไปด้วยความสัมพันธ์แบบหลายต่อหลาย วิธีที่เป็นธรรมชาติที่สุดคือมองเป็นกราฟ

กราฟประกอบด้วยสองอย่าง อย่างแรกคือจุด หรือ vertex ซึ่งแทนสิ่งต่าง ๆ อย่างที่สองคือเส้นเชื่อม หรือ edge ซึ่งแทนความสัมพันธ์ระหว่างสิ่งเหล่านั้น

ตัวอย่างในชีวิตจริงเช่น

- เครือข่ายสังคม จุดคือคน เส้นเชื่อมคือความเป็นเพื่อน
- เว็บไซต์ จุดคือหน้าเว็บ เส้นเชื่อมคือลิงก์
- แผนที่ถนน จุดคือทางแยก เส้นเชื่อมคือถนน

ที่น่าสนใจคือ กราฟเดียวเก็บข้อมูลต่างชนิดกันได้ เช่น Facebook ใช้กราฟเดียวเก็บทั้งคน สถานที่ เหตุการณ์ การเช็กอิน และความคิดเห็น

ตัวอย่างหลักในหนังสือคือ ลูซี่ เกิดที่รัฐไอดาโฮ ประเทศสหรัฐอเมริกา และปัจจุบันอาศัยอยู่ที่ลอนดอน ประเทศอังกฤษ ส่วนอแลง เกิดที่ฝรั่งเศส ทั้งสองแต่งงานกัน ในกราฟ สถานที่ซ้อนกันเป็นชั้น ๆ คือ ไอดาโฮ อยู่ในสหรัฐอเมริกา ซึ่งอยู่ในอเมริกาเหนือ

โมเดล Property Graph

ในโมเดลนี้ ทั้งจุดและเส้นเชื่อมมีคุณสมบัติติดตัวได้ เช่น จุดที่เป็นคนมีชื่อ ส่วนเส้นเชื่อมมีป้ายบอกว่าเป็นความสัมพันธ์แบบไหน เช่น เกิดที่ อาศัยอยู่ที่ หรือแต่งงานกับ

จุดเด่นคือยืดหยุ่นมาก ไม่ได้บังคับว่าอะไรเชื่อมกับอะไรได้บ้าง ระดับของสถานที่ก็ต่างกันได้ในแต่ละประเทศ บางประเทศแบ่งเป็นรัฐ บางประเทศแบ่งเป็นจังหวัด กราฟรองรับได้ทั้งหมด และถ้าวันหนึ่งอยากเพิ่มข้อมูลแบบใหม่ เช่น อาหารที่แต่ละคนแพ้ ก็เพิ่มได้ทันที ไม่ต้องรื้อของเดิม

ภาษา Cypher

Cypher คือภาษาสืบค้นแบบประกาศของฐานข้อมูลกราฟชื่อ Neo4j

สมมติเราอยากหาว่า มีใครบ้างที่เกิดในสหรัฐอเมริกา แต่ตอนนี้อาศัยอยู่ในยุโรป ใน Cypher เราเขียนรูปแบบเส้นทางที่ต้องการ ว่าคนคนนั้นต้องมีเส้นทางเกิดที่ ไล่ขึ้นไปจนถึงสหรัฐอเมริกา และมีเส้นทางอาศัยอยู่ที่ ไล่ขึ้นไปจนถึงยุโรป ไม่ว่าจะต้องผ่านกี่ชั้นก็ตาม คำสั่งนี้ยาวแค่สี่บรรทัด

ถ้าใช้ SQL ทำเรื่องเดียวกัน

ทำได้ แต่ยากกว่ามาก เพราะเราไม่รู้ล่วงหน้าว่าต้องไล่ผ่านกี่ชั้น บางคนเกิดในเมือง ซึ่งอยู่ในรัฐ ซึ่งอยู่ในประเทศ บางคนอาจระบุแค่ประเทศเลย

SQL รุ่นใหม่รองรับเรื่องนี้ด้วยคำสั่งชื่อ WITH RECURSIVE แต่คำสั่งเดียวกันต้องเขียนยาวถึงประมาณ 29 บรรทัด ตัวอย่างนี้แสดงให้เห็นว่าการเลือกโมเดลข้อมูลให้เหมาะกับงาน มีผลต่อความง่ายของงานมากแค่ไหน

โมเดล Triple Store และภาษา SPARQL

อีกวิธีหนึ่งคือเก็บข้อมูลเป็นชุดละสามส่วน คือ ประธาน กริยา และกรรม เช่น ลูซี่ อายุ 33 ปี หรือ ลูซี่ แต่งงานกับ อแลง

แนวคิดนี้เกี่ยวข้องกับโครงการ Semantic Web ซึ่งเคยตั้งเป้าให้เว็บไซต์ทั่วโลกเผยแพร่ข้อมูลในรูปแบบที่คอมพิวเตอร์อ่านเข้าใจได้ โดยใช้มาตรฐานชื่อ RDF โครงการนี้ไม่ได้ประสบความสำเร็จอย่างที่หวัง แต่แนวคิดและเครื่องมือหลายอย่างยังมีประโยชน์

ภาษาสำหรับสืบค้นข้อมูลแบบนี้ชื่อ SPARQL หน้าตาคล้าย Cypher และเขียนได้กระชับเช่นกัน

ภาษา Datalog

Datalog เป็นภาษาเก่าแก่ที่เป็นรากฐานของภาษาสืบค้นหลายตัว แนวคิดคือเขียนกฎเล็ก ๆ ทีละข้อ แล้วนำกฎมาต่อกันเป็นกฎที่ซับซ้อนขึ้น คล้ายการแตกฟังก์ชันใหญ่ออกเป็นฟังก์ชันย่อย ๆ ระบบที่ใช้ Datalog เช่น Datomic

สำหรับงานง่าย ๆ ภาษานี้อาจดูยุ่งยากกว่าภาษาอื่น แต่เมื่อข้อมูลซับซ้อนมาก การประกอบกฎเข้าด้วยกันจะช่วยได้มาก

ข้อสังเกตเรื่องกราฟกับโมเดลเครือข่ายในอดีต

บางคนอาจคิดว่ากราฟก็คือโมเดลเครือข่ายสมัยก่อนที่กลับมาใหม่ แต่จริง ๆ ต่างกันมาก กราฟไม่ได้จำกัดว่าอะไรเชื่อมกับอะไรได้ เราเข้าถึงจุดไหนก็ได้โดยตรง ไม่ต้องไล่ตามเส้นทาง และที่สำคัญ กราฟสมัยใหม่มีภาษาแบบประกาศอย่าง Cypher และ SPARQL ให้ใช้

ตอนที่ 5: สรุปท้ายบท

มาทบทวนกันอีกครั้งครับ

หนึ่ง ในอดีต ข้อมูลเคยถูกเก็บเป็นต้นไม้ใหญ่ต้นเดียว แต่แบบนี้รองรับความสัมพันธ์แบบหลายต่อหลายได้ไม่ดี โมเดลเชิงสัมพันธ์จึงเกิดขึ้นมาแก้ปัญหา

สอง ต่อมา มีแอปพลิเคชันจำนวนมากที่ไม่เข้ากับโมเดลเชิงสัมพันธ์ จึงเกิดฐานข้อมูล NoSQL ซึ่งแยกไปสองทาง ทางแรกคือฐานข้อมูลเอกสาร เหมาะกับข้อมูลที่อยู่เป็นก้อน ๆ และไม่ค่อยเชื่อมโยงกัน ทางที่สองคือฐานข้อมูลกราฟ เหมาะกับข้อมูลที่ทุกอย่างอาจเชื่อมโยงถึงกันได้

สาม แต่ละโมเดลมีข้อดีของตัวเอง และไม่มีโมเดลไหนดีที่สุดสำหรับทุกงาน สิ่งที่ทำได้ง่ายในโมเดลหนึ่ง อาจยากมากในอีกโมเดลหนึ่ง

สี่ ทั้งฐานข้อมูลเอกสารและกราฟ มักไม่บังคับโครงสร้างข้อมูลตั้งแต่ตอนเขียน ทำให้ปรับเปลี่ยนตามความต้องการใหม่ได้ง่ายกว่า แต่ความจริงแล้วโครงสร้างก็ยังมีอยู่ เพียงแต่ไปตีความตอนอ่าน

ห้า ภาษาแบบประกาศ เช่น SQL Cypher และ SPARQL มักดีกว่าการสั่งทีละขั้น เพราะสั้นกว่า เข้าใจง่ายกว่า และเปิดให้ระบบปรับปรุงประสิทธิภาพได้เอง

บทต่อไปคือบทที่สาม เราจะลงลึกไปดูว่า ฐานข้อมูลเก็บข้อมูลลงดิสก์และค้นหาข้อมูลอย่างไร ซึ่งเป็นเรื่องที่สนุกมากครับ ขอบคุณที่ฟังครับ